PrepAce: End-to-End Technical Interview Guide

EXAM-READY COMPREHENSIVE ARCHITECTURE & DEEP DIVE Q&A; COMPENDIUM

---

1. Highest-Probability Core System Questions

Q: Explain your PrepAce project end-to-end.

PrepAce is an AI-powered, speech-first mock interview platform designed for candidate interview preparation. It ingests candidate resumes, extracts structured profile data, generates tailored initial interview question pools, and executes dynamic multi-turn voice interviews. During the interview, candidate verbal responses are transcribed, evaluated in real time by an AI engine for correctness, depth, relevance, and clarity, and followed up dynamically. Upon session completion, the platform aggregates scores deterministically and delivers an executive candidate performance dashboard with actionable AI feedback.

> ***Key Interview Takeaway:*** *PrepAce combines a React frontend (Web Speech API for audio/TTS), a Spring Boot backend (business domain, JWT, state, PostgreSQL), and a Python FastAPI AI engine (Groq LLaMA 3.3 70B).*

Q: What problem does it solve?

Traditional technical interview preparation platforms (like LeetCode or static Q&A; banks) focus purely on written code execution without evaluating spoken technical communication, architectural reasoning, or adaptive follow-up handling. Human mock interviews are expensive, non-scalable, and scheduling-constrained. PrepAce bridges this gap by providing an on-demand, adaptive, voice-first AI interviewer that simulates real technical interview dynamics with objective feedback.

> ***Key Interview Takeaway:*** *Solves non-scalable human interview costs while providing dynamic voice interaction and objective, reproducible scoring.*

Q: What happens from resume upload to final interview result?

The workflow follows 5 distinct phases:
1. **Resume Upload & Analysis:** Candidate uploads PDF resume → React sends file to Spring Boot → Spring Boot forwards to FastAPI (`POST /analyze`) → Groq LLaMA extracts structured CandidateProfile (skills, frameworks, projects) → Saved in PostgreSQL.
2. **Question Pool Generation:** User starts interview → Spring Boot calls FastAPI (`POST /generate-interview-questions`) using CandidateProfile + target role → Groq returns 5 structured candidate questions.
3. **Voice Room Interaction:** Candidate enters room → Browser TTS speaks question → Candidate answers verbally → Web Speech API transcribes answer to text.
4. **Dynamic Evaluation & Next Question:** Answer submitted → Spring Boot calls FastAPI (`POST /evaluate-and-next-question`) in **1 single LLM call** → Returns answer score/feedback AND evaluates whether to ask a follow-up or transition topic.
5. **Deterministic Results Aggregation:** Upon session finish, Spring Boot calculates deterministic weighted scores (Accuracy 35%, Depth 25%, Relevance 20%, Clarity 20%) and requests a single qualitative summary from FastAPI (`POST /generate-final-feedback`).

Key Interview Takeaway: *End-to-end data flows synchronously across React -> Spring Boot -> FastAPI -> Groq LLaMA with full state persistence in PostgreSQL.*

Q: Why did you use three separate layers: React, Spring Boot, and FastAPI?

Each layer enforces clear Separation of Concerns (SoC) based on framework strengths:
• **React (Frontend):** Handles rich UI state,Web Speech API audio recognition/synthesis, state machine transitions, and responsive dark Cyber-Lime dashboard rendering.
• **Spring Boot (Backend Domain):** Enterprise-grade security (JWT/Spring Security), ACID transactional database operations, optimistic locking, idempotent state management, and business workflow enforcement.
• **FastAPI (AI Microservice):** High-performance asynchronous Python service dedicated exclusively to AI model orchestration, prompt template formatting, PyPDF2 parsing, and Groq LLM integration.

Key Interview Takeaway: *Spring Boot handles relational state & security; FastAPI handles AI ecosystem integration; React handles real-time user voice interaction.*

Q: Why not build the entire backend in Spring Boot? Why not build everything in FastAPI?

Building everything in Spring Boot would force Python-native AI libraries (like PyPDF2, LangChain, or direct Pydantic JSON validation) into heavy Java wrappers or Jython, introducing architectural complexity. Conversely, building everything in FastAPI would compromise enterprise relational capabilities (Hibernate/JPA entity management, robust migration tooling, spring-security JWT filters, enterprise transaction propagation). Separating them allows Spring Boot to remain the immutable source of state truth while FastAPI serves as a stateless AI engine.

Key Interview Takeaway: *Polyglot architecture leverages Java for enterprise reliability and Python for AI ecosystem velocity.*

Q: How does your dynamic interview work? How does the next question depend on the candidate's previous answer?

The interview engine evaluates candidate answers against target criteria (correctness, depth, relevance, clarity). If a candidate provides a **Strong** answer, the engine increases difficulty or moves to a higher-level system design follow-up. If the answer is **Partial**, the engine asks a targeted clarifying follow-up on missing details. If the answer is **Incorrect** or the candidate says **'I don't know'**, the engine avoids repetitive probing on that subtopic and seamlessly transitions to the next queued topic.

Key Interview Takeaway: *Evaluations dynamically adjust follow-up depth up to a maximum threshold (e.g. 2 follow-ups) before enforcing topic switches.*

Q: How do you minimize latency? Why combine answer evaluation + next-question generation into one LLM call?

In a real-time voice interview, latency is critical. Executing separate LLM calls for evaluation and next-question generation would double network round-trips to ~2–3 seconds total. By combining evaluation AND next-question selection into a **single LLM prompt call (`POST /evaluate-and-next-question`)**, latency is reduced to ~600ms–800ms total. Exactly **1 LLM call occurs per interview turn**.

Key Interview Takeaway: *Single-call turn execution cuts API network latency in half and keeps turn times under 1 second.*

Q: How does resume-based question generation work? Why CandidateProfile?

During resume parsing, PyPDF2 extracts raw text, which Groq structures into a `CandidateProfile` entity containing languages, frameworks, projects, and impact metrics. Instead of sending raw PDF resume text (~5,000 tokens) on every single interview turn, the backend stores and sends the compact structured `CandidateProfile` JSON (~300 tokens). This reduces prompt token usage by ~90% and ensures questions directly target candidate experience.

Key Interview Takeaway: *CandidateProfile acts as a token-efficient snapshot of the candidate's background.*

Q: Which LLM are you using and why Llama 3.3 70B through Groq? What happens if Groq fails?

We use **Llama-3.3-70b-versatile** hosted on **Groq LPU (Language Processing Unit)** infrastructure. Groq provides sub-second inference speeds (~280 tokens/sec), essential for voice interactivity. If Groq API calls fail or timeout (8s limit), `FastApiAiEngineClient` automatically fails over to `PlaceholderAiEngineClient`, which generates deterministic fallback evaluations and standard next questions without crashing the candidate's session.

Key Interview Takeaway: *Groq LPU guarantees sub-second response times; Java client abstraction guarantees zero interview downtime via fallback execution.*

Q: How does your voice interview work? Web Speech API vs Whisper? STT vs TTS?

• **STT (Speech-To-Text):** Web Speech API (`window.SpeechRecognition`) transcribes candidate spoken audio to text locally in the browser.
• **TTS (Text-To-Speech):** Web Speech API (`window.speechSynthesis`) converts interviewer question text into audio spoken by the browser.
• **Transcripts vs Audio:** Sending transcripts to the backend instead of raw audio files saves bandwidth, eliminates backend audio chunk processing, and reduces payload size from megabytes to bytes.
• **Web Speech API vs Whisper:** Web Speech API is used for real-time streaming in the browser with 0 server cost; Whisper is available for offline audio file parsing.

Key Interview Takeaway: *Client-side STT/TTS eliminates audio streaming infrastructure costs and minimizes latency.*

Q: What happens when the candidate says 'I don't know'?

When the candidate explicitly states 'I don't know' or skips, the system bypasses unnecessary LLM deep-probing. The answer is recorded with zero score, the evaluation notes the skip cleanly without generating fake praise, and the engine immediately transitions to the next primary topic from the question pool.

Key Interview Takeaway: *Prevents awkward AI probing on unanswerable topics and maintains candidate session momentum.*

2. Spring Boot & Backend Architecture Questions

Q: Explain the entities and relationships in your interview system.

The relational model consists of 5 core entities:
• `User` (1) ■■ (N) `InterviewSession`: User owns multiple sessions.
• `InterviewSession` (1) ■■ (N) `InterviewQuestion`: Session contains queued/asked questions.
• `InterviewQuestion` (1) ■■ (1) `CandidateAnswer`: Each question has at most one candidate answer.
• `CandidateAnswer` (1) ■■ (1) `QuestionEvaluation`: Each answer has one evaluation.
• `InterviewSession` (1) ■■ (1) `InterviewResult`: One finalized result per session.

Key Interview Takeaway: *Relational integrity is maintained via JPA `@ManyToOne` and `@OneToOne` associations with FK constraints.*

Q: Why is InterviewSession your root entity and what is its lifecycle?

`InterviewSession` acts as the Aggregate Root in Domain-Driven Design (DDD). All state transitions, question indices, and results attach to a session. Its lifecycle transitions are: `CREATED` → `IN_PROGRESS` → `COMPLETED` (or `ABANDONED`). Accessing a result for an uncompleted session yields HTTP 409 Conflict.

Key Interview Takeaway: *Aggregate root pattern prevents inconsistent state mutations across interview sub-entities.*

Q: What does @Transactional do and why use @Version optimistic locking?

• `@Transactional` ensures that answer persistence, evaluation mapping, and question index increments occur atomically in PostgreSQL. If any step fails, the entire transaction rolls back.
• `@Version` Optimistic Locking prevents race conditions if two duplicate answer submission requests hit the server simultaneously. Hibernate checks the version column; the second concurrent write throws an `OptimisticLockException` and is rejected cleanly.

Key Interview Takeaway: *Atomicity prevents partial state persistence; optimistic locking prevents double-submission race conditions.*

Q: How do you ensure one user cannot access another user's interview? (JWT Auth vs Authorization)

Spring Security extracts and validates the JWT Bearer token on every request (`JwtAuthenticationFilter`), placing the authenticated `UserId` in the `SecurityContext`. In `InterviewSessionService`, every fetch query enforces ownership: `session.getUser().getId().equals(currentUserId)`. If ownership fails, the server throws `UnauthorizedAccessException`, mapped by `GlobalExceptionHandler` to HTTP 404 NOT_FOUND to prevent resource enumeration leaks.

Key Interview Takeaway: *Returns 404 instead of 403 on unauthorized access to prevent attackers from probing valid Session UUIDs.*

Q: How does Spring Boot communicate with FastAPI? What is the AiEngineClient abstraction?

Spring Boot uses `RestClient` (or `RestTemplate`) with explicit timeouts (8s read timeout) to send JSON payloads to FastAPI (`http://localhost:8000`). `AiEngineClient` is a Java interface defining AI operations. We implement two classes:
1. `FastApiAiEngineClient`: Primary production client invoking FastAPI endpoints.
2. `PlaceholderAiEngineClient`: Fallback implementation performing deterministic calculations.
Spring Boot is the immutable source of truth for interview state; FastAPI is purely a stateless processing engine.

Key Interview Takeaway: *Interface abstraction enables seamless failover and decoupled testing without mock HTTP servers.*

3. AI / LLM Architecture & Deterministic Scoring

Q: Is the final numerical score generated by the LLM? How is it calculated?

NO. The final numerical score is NOT generated by the LLM.
Individual turn evaluations score Correctness, Depth, Relevance, and Clarity (0–100). Upon completion, Spring Boot deterministically aggregates all evaluations stored in PostgreSQL using the exact weighted formula:
$$\text{Overall} = (\text{Correctness} \times 0.35) + (\text{Depth} \times 0.25) + (\text{Relevance} \times 0.20) + (\text{Clarity} \times 0.20)$$
Scores are rounded to 1 decimal place. The LLM is invoked **at most once at the end** solely for qualitative executive feedback.

Key Interview Takeaway: *Deterministic mathematical aggregation guarantees 100% reproducible, non-hallucinated scoring.*

Q: Why is deterministic scoring preferable to asking the LLM for the final score?

1. **Reproducibility & Fairness:** LLMs exhibit variance across calls. Asking an LLM for a final score could yield 82% on one call and 74% on another for the exact same transcript.
2. **Cost & Latency:** Eliminates sending the entire 10-turn conversation history back to the LLM.
3. **Zero Hallucination:** Mathematical formulas cannot hallucinate non-existent scores or ignore un-answered questions.

Key Interview Takeaway: *Guarantees strict auditability, cost savings, and zero scoring variance.*

Q: How do you force the LLM to return structured JSON? What if it returns malformed JSON?

We enforce JSON formatting using Pydantic v2 schemas in FastAPI and system prompt constraints (`JSON ONLY, NO MARKDOWN`). FastAPI parses response strings using `json.loads()`. If malformed JSON or markdown code blocks (` ```json `) are returned, FastAPI strips markdown block syntax and validates fields against Pydantic models. If validation fails, fallback DTOs are returned safely.

Key Interview Takeaway: *Robust Pydantic parsing and regex fallback handling protect against raw string LLM output errors.*

Q: How would you reduce AI cost if you had 100,000 users?

1. **Prompt Compression & Caching:** Cache common system prompts and static role rubrics in Redis.
2. **Smaller Models for Easy Questions:** Route initial screening questions to Llama 3b/8b models and reserve 70B for complex evaluations.
3. **Deterministic Local Rule Engine:** Bypasses LLM calls entirely for empty or short 'I don't know' answers.
4. **Batch Summary Feedback:** Queue final feedback generation in background job workers.

Key Interview Takeaway: *Reduces token count by ~80% and inference costs by ~90% at scale.*

4. Speech & React Frontend State Architecture

Q: Explain the Voice Interview Room state machine.

The voice room is managed by an explicit finite state machine (FSM):
1. `QUESTION_LOADING`: Fetching question from backend.
2. `INTERVIEWER_SPEAKING`: Browser TTS speaking question (Mic disabled to prevent loopback feedback).
3. `READY`: TTS finished; waiting for candidate to initiate response.
4. `LISTENING`: STT active, capturing audio input.
5. `TRANSCRIPT_READY`: Candidate finalized response; transcript ready for review.
6. `SUBMITTING`: Submitting transcript to backend.
Using an explicit FSM prevents invalid state overlap (e.g. mic listening while speaker is talking).

Key Interview Takeaway: *FSM prevents microphone self-echo and eliminates race conditions present in boolean flag setups.*

Q: How do custom hooks useSpeechRecognition and useSpeechSynthesis work?

• `useSpeechRecognition`: Encapsulates `window.SpeechRecognition`, managing event listeners (`onresult`, `onerror`, `onend`), interim transcript buffering, and auto-restart logic.
• `useSpeechSynthesis`: Encapsulates `window.speechSynthesis`, controlling voice selection, utterance queueing, speaking status tracking, and clean unmount cancellation via `window.speechSynthesis.cancel()`.

Key Interview Takeaway: *Encapsulates browser API event listeners and ensures clean memory unmounting on route changes.*

5. PostgreSQL Database & Persistence

Q: What are N+1 queries, and could your entity relationships cause them?

An N+1 query problem occurs when fetching a parent entity (e.g. 10 `InterviewSession` records) causes Hibernate to execute 1 query for parents plus N separate queries for children (`User` or `CandidateAnswer`). In PrepAce, we prevent N+1 queries by configuring `@ManyToOne(fetch = FetchType.LAZY)` on relationships and utilizing explicit JPQL `JOIN FETCH` queries when loading sessions with answers and evaluations.

Key Interview Takeaway: *LAZY fetching and JOIN FETCH queries optimize SQL execution to 1 single database query.*